Contents

Lecture Notes: Causal Forest (L13 — Data Science Version)

Course: Causal Inference for Data Science
Slides file: `L13_CausalForest_data_v5.tex`
Audience: Graduate students in economics (data-science track)
Language: English

Table of Contents

Main Idea

[Slide] Title — Main Idea

Good morning. Today we begin a new lecture on **causal forest** — one of the most important machine-learning methods that has entered the toolkit of empirical economists.

Before we dive in, let me briefly place this lecture in the context of the course. We have spent the past several weeks studying identification strategies: randomized controlled trials, difference-in-differences, regression discontinuity design, and instrumental variables. All of these methods were designed to give a credible estimate of one number — the **average treatment effect (ATE)** or, sometimes, the ATT.

Today's question is different. We are not going to challenge those identification strategies. Instead, we are going to ask: *given* that we have a credible causal estimate, can we go further and ask **for whom** the treatment works?

This is the central question of causal forest.

[Slide] Treatment Effect Heterogeneity

Let me start with the formal setup. In previous lectures, we focused on the **Average Treatment Effect**:

$$\alpha_{\text{ATE}} = \text{E}[Y_i^1 - Y_i^0]$$

This is an average over the entire population. It is a useful summary, but it hides a lot of information. In particular, it tells us nothing about whether some groups benefit more than others.

The object we want to estimate today is the **Conditional Average Treatment Effect (CATE)**:

$$\tau(x) = \text{E}[Y_i^1 - Y_i^0 \mid X_i = x]$$

Notice that $\tau(x)$ is a *function* of observed characteristics x. It tells us the average treatment effect for workers who share the same observed characteristics x.

Why does this matter for policy? Consider a job training program. We might find that the ATE is a modest \$1,500. But if young, low-education workers gain \$4,000 while older, high-experience workers gain almost nothing, then the *targeting* of the program becomes critically important. If the government has a limited budget, it should prioritize the workers for whom the program works best.

Causal forest is the tool that estimates $\tau(x)$.

[Slide] Motivating Example: Job Training

Let us make this concrete. Suppose a government offers a job training program.

- **Outcome** Y_i: earnings after the program.

- **Treatment** D_i: whether worker i participated.
- **Covariates** X_i: age, education, race, marital status, prior earnings.

The main policy question is: **does the training help everyone equally, or does it help some workers more than others?**

This is not just an academic question. If the training budget is limited, the government needs to know who benefits most.

Causal forest gives us a principled, data-driven way to answer this.

[Slide] Why Heterogeneity Matters (table)

Look at the table on the slide. Three groups of workers face the same training program but experience very different treatment effects:

- **Young, low education**: \$8,000 without training → \$11,000 with training. Effect: \$3,000.
- **Older, high experience**: \$18,000 → \$18,500. Effect: only \$500.
- **Young, low experience**: \$2,000 → \$6,000. Effect: \$4,000.

If we average these effects together, we get something moderate — perhaps around \$2,500. That average is not wrong, but it obscures something important.

Policy decisions often require knowing **where** the effect is large. If resources are limited, should the government target young, low-experience workers? The ATE does not answer this. CATE does.

[Slide] Why Heterogeneity Matters (policy examples)

Let me give you a broader sense of why heterogeneity matters across labor economics.

- **Youth employment programs**: different youths face different labor market conditions. The program may help workers in depressed local labor markets but not those in tight markets.
- **Education policy**: returns to schooling may differ by family background, local school quality, or access to credit.
- **Minimum wage**: firms in low-wage, low-skill industries may respond very differently than firms in higher-skill sectors.
- **Parental leave**: the effect on career outcomes may depend strongly on previous earnings and occupation type.

In each of these cases, a single ATE estimate is useful, but knowing *who* benefits most allows for smarter policy design.

[Slide] Treatment Effect Heterogeneity — Traditional Approaches

How have economists traditionally explored heterogeneity? Two main approaches:

1. Theory-guided subgroup analysis. The researcher picks subgroups based on economic theory — men vs. women, young vs. old, educated vs. uneducated — and estimates the ATE within each group.

The problem: this approach requires the researcher to *pre-specify* which characteristics matter. If you check enough subgroups, some will look significant by chance. And interactions between multiple characteristics are hard to analyze this way.

2. Interaction terms in regression:

$$Y_i = \alpha + \beta D_i + \gamma X_i + \delta \left(D_i \times X_i\right) + \varepsilon_i$$

The coefficient δ captures how the treatment effect varies with X_i.

The problem: this assumes a **linear and parametric** relationship between X_i and the treatment effect. If the true relationship is nonlinear, or involves interactions between multiple covariates, this approach will miss it.

[Slide] Causal Forest: A Data-Driven Approach (1)

Causal forest takes a different approach. Instead of pre-specifying which variables matter and assuming a linear form, it estimates the entire function $\tau(x)$ in a data-driven, nonparametric way.

The key idea is shown in the diagram:

$$\text{Data } (Y, D, X) \;\longrightarrow\; \text{Many honest causal trees} \;\longrightarrow\; \text{CATE } \widehat{\tau}(x)$$

We feed in the data. The algorithm builds many trees, each trained on a different random subsample and using a random subset of covariates. Averaging across trees gives a stable, flexible estimate of $\tau(x)$.

The word "honest" is important — we will come back to it when we discuss causal trees in detail.

[Slide] Causal Forest: A Data-Driven Approach (2)

I want to emphasize one critical point: **causal forest is not an identification strategy**.

It does not solve the selection bias problem on its own. You cannot use causal forest to turn observational data into a causal analysis without additional assumptions.

What causal forest *is*, is a flexible **estimator** that can be combined with any identification strategy:

- **RCT**: who benefits more from the randomly assigned treatment?
- **DID**: are treatment effects heterogeneous across groups or time periods?
- **RDD**: who near the cutoff benefits most?

The research design provides causal credibility. Causal forest provides flexibility in how we describe heterogeneity, without requiring the researcher to pre-specify which variables matter.

[Slide] How ATE and ATT Relate to CATE

One last conceptual point before we get into the mechanics. Causal forest does not *replace* the ATE or ATT. It estimates a richer object — the CATE function — from which the familiar estimands are recovered by averaging:

$$\alpha_{\text{ATE}} = \mathrm{E}[\tau(X_i)], \qquad \alpha_{\text{ATT}} = \mathrm{E}[\tau(X_i) \mid D_i = 1]$$

So causal forest first estimates $\hat{\tau}(x_i)$ for every worker, and then computes averages over the relevant population.

This has an important implication: if treatment effects are actually homogeneous — if every worker benefits by the same amount — then $\tau(x) \approx \alpha_{\text{ATE}}$ for all x, and causal forest collapses back to the usual ATE estimator. Causal forest is a *generalization*, not a replacement.

Decision Tree and Random Forest

[Slide] Section Title — Decision Trees

Let me now build up the machinery. We will start with **decision trees**, then see why a single tree is not enough, introduce **random forests**, and finally arrive at **causal forests** step by step.

[Slide] Decision Trees — Main Idea

A **decision tree** is a prediction algorithm that works by recursively partitioning the covariate space into rectangular regions, and then predicting the mean of Y within each region.

The algorithm is called **recursive binary splitting**:

1. Find the variable X^j and the split point s that minimizes prediction error.
2. Divide the data into two regions: $\{X^j < s\}$ and $\{X^j \geq s\}$.
3. Repeat the same procedure inside each region until a stopping rule is satisfied.

The final regions are called **leaves** or **terminal nodes**. Within each leaf, the prediction is simply the mean of Y for all observations that land in that leaf.

Decision trees are attractive because they are flexible — they can capture nonlinear relationships and interactions without specifying a functional form. They are also interpretable: you can follow the splits to understand what the model is doing.

[Slide] Decision Trees — Finding the Best Split

Let me spell out the criterion for choosing the best split. At each step, we evaluate every possible combination of variable X^j and split point s by computing the **residual sum of squares (RSS)** after the split:

$$\text{RSS}(j, s) = \sum_{i \in R_1} (Y_i - \bar{Y}_{R_1})^2 + \sum_{i \in R_2} (Y_i - \bar{Y}_{R_2})^2$$

where $R_1 = \{i : X_i^j < s\}$ and $R_2 = \{i : X_i^j \geq s\}$, and $\bar{Y}_{R_1}, \bar{Y}_{R_2}$ are the means within each region.

We choose the combination (j^*, s^*) that gives the **smallest** RSS. Intuitively, this finds the split that makes observations within each region most similar to each other in terms of the outcome Y.

We then repeat this process inside each resulting region, and so on recursively.

[Slide] Decision Trees — When to Stop Splitting (1)

Without a stopping rule, a tree that keeps splitting until every leaf contains exactly one observation would perfectly fit the training data. That is **overfitting** — the tree has memorized the data rather than learned the true pattern, and it will perform poorly on new data.

Two common stopping rules:

Minimum leaf size. Stop splitting if a region contains fewer than k observations (for example, $k = 5$). This ensures each leaf has enough data to compute a reliable estimate of $\bar{Y}$.

Maximum tree depth. Halt after d levels of recursive splits (for example, $d = 5$). This limits the total number of leaves to at most 2^d, keeping the tree manageable.

[Slide] Decision Trees — When to Stop Splitting (2)

A third rule:

Minimum RSS reduction. Stop splitting a region if the best available split reduces RSS by less than ε. This skips splits that add complexity without meaningfully improving fit.

These stopping rules are called **hyperparameters** because the researcher must choose them. In the `grf` package, the key hyperparameter is `min.node.size`, which controls the minimum leaf size.

The deeper message: every machine learning method has hyperparameters that control the bias-variance tradeoff. A deeper tree has lower bias (it fits the training data better) but higher variance (it is sensitive to noise). A shallower tree has higher bias but lower variance. We want the sweet spot.

[Slide] Decision Trees — Illustration

To make this concrete: suppose we want to predict wages Y using age and education.

- **Split 1**: age < 30 vs. age ≥ 30.
- **Split 2** (within age ≥ 30): education < 12 vs. education ≥ 12.

This produces three leaves:

- Leaf 1: young workers (age < 30), predicted wage $\hat{Y} = \$8,200$.
- Leaf 2: older workers, low education (age ≥ 30, educ < 12), predicted wage $\hat{Y} = \$10,100$.

- Leaf 3: older workers, higher education (age ≥ 30, educ ≥ 12), predicted wage $\hat{Y} = \$11,500$.

Notice: all workers in Leaf 1 get the same predicted wage $\$8,200$, regardless of their exact age or education. The tree approximates the true relationship in a step-function manner.

[Slide] Decision Trees — Tree Structure Example

The slide shows the tree diagram graphically. Starting at the root:

- **Root node**: Is age < 30? Yes → Leaf 1 ($\hat{Y} = \$8,200$). No → go right.
- **Second node**: Is educ < 12? Yes → Leaf 2 ($\hat{Y} = \$10,100$). No → Leaf 3 ($\hat{Y} = \$11,500$).

Prediction for a new worker with age 35 and education 14: follow the right branch (age ≥ 30), then the right branch again (educ ≥ 12), landing in **Leaf 3** with prediction $\$11,500$.

The orange nodes are decision rules. The green nodes are predictions. Reading the tree from root to leaf is how you trace the prediction for any new observation.

[Slide] Decision Trees — Partition of Feature Space

The right panel shows what the tree does in the feature space (age on the x-axis, education on the y-axis).

- The vertical line at age $= 30$ is **Split 1**.
- The horizontal line (in the right half only) at educ $= 12$ is **Split 2**.

The result is three rectangular regions, colored differently. Every observation within the same rectangle gets the same predicted wage.

Key insight: the tree partitions the feature space into **rectangles**. A deeper tree creates more splits, finer rectangles, and a more flexible approximation. But as we discussed, there is a price: overfitting.

The tree approximates any nonlinear relationship between Y and X without specifying a functional form — as long as the tree is deep enough but not too deep.

Random Forest

[Slide] Section Title — Random Forest

Decision trees are elegant and interpretable, but they suffer from a fundamental problem: **high variance**. We now build to the solution: the **random forest**.

[Slide] Why a Single Decision Tree Is Not Enough

A single deep tree has a crucial weakness. Because it "memorizes" the training data, **small changes in the data can lead to a completely different tree structure**.

Predictions on new data are therefore unstable. In statistical terms, the tree has low bias (it fits training data well) but high variance (it is sensitive to which particular observations happen to be in the training sample).

The intuition: asking one person for an opinion is noisy. Asking 100 independent people and averaging their answers is much more reliable. Random forest does the same thing with trees.

[Slide] High Variance: Sample A vs. Sample B

Let me show you the variance problem with a concrete example. We have five workers and want to predict wages using age and education.

Sample A has workers at ages {20, 25, 40, 45, 50} and education levels {9, 9, 9, 9, 12}. The wages range from \$8 to \$23. Older workers earn more — so the tree splits on **Age**.

Sample B is almost identical, but row 2 changes: the 25-year-old with 9 years of education and \$9/hr is replaced by a 25-year-old with **12 years of education and \$30/hr** (highlighted in red on the slide).

This single replacement completely changes the picture for young workers: now there is one young worker earning \$8 and another earning \$30, making age a noisy predictor within the young group. The tree switches to splitting on **Education**.

[Slide] High Variance: RSS Table

Why does the tree switch? Because it always picks the split with the lowest RSS.

Candidate split	RSS in Sample A	RSS in Sample B
Age < 30	**2.5**	244
Educ < 11	170	**147**

In Sample A, age splits the data cleanly (young: low wages, old: high wages) → RSS = 2.5. In Sample B, that one high-wage young worker makes the age split terrible → RSS jumps to 244. Meanwhile, education now separates high-wage from low-wage workers better → RSS = 147.

The lesson: **replacing just one observation causes the tree to choose a completely different split variable**. This is high variance in action.

[Slide] High Variance: Tree A vs. Tree B

The visual shows the two trees side by side:

- **Tree A** (from Sample A): splits on **Age** < 30**?** → leaves: \$8.5/hr and \$22/hr.
- **Tree B** (from Sample B): splits on **Educ** < 11**?** → leaves: \$17/hr and \$26.5/hr.

These two trees look completely different. One changed observation flipped the entire tree structure.

[Slide] High Variance: Prediction Gap

Now suppose a new worker arrives: age = 22, education = 16 years.

Tree	Decision path	Predicted wage
Tree A	Age $< 30 \rightarrow$ Yes	\$8.5/hr
Tree B	Educ $< 11 \rightarrow$ No	\$26.5/hr

Same worker, same features — yet the two trees disagree by almost \$18/hr.

This is the variance problem made visible. The solution is not to choose between Tree A and Tree B, but to **grow many diverse trees and average their predictions**.

That is the random forest.

[Slide] Random Forest: Two Sources of Randomness

A random forest makes each tree different from all others in **two ways**:

1. Random subsampling. Each tree is trained on a random subset of observations (drawn with or without replacement). Tree 1 sees a different set than Tree 2, which sees a different set than Tree B. Each tree therefore learns a slightly different version of the pattern.

2. Random feature selection. At each candidate split, the tree considers only m randomly selected covariates out of the total p. A typical choice is $m = \sqrt{p}$. This prevents all trees from using the same strongest variable at every split. With 9 covariates (as in our R example), each split considers about 3 randomly chosen variables.

The result: trees are **useful but not identical**. Their errors are not perfectly correlated, so when we average them, the errors partly cancel out.

The boxed message on the slide captures this: "trees are useful but not identical $\rightarrow$ their errors partly cancel when averaged."

[Slide] Random Forest: Algorithm

For a new observation with covariates x:

1. Grow B different decision trees (e.g., $B = 2000$). Each tree uses a random subsample and a random feature subset at each split.
2. Each tree produces its own prediction:

$$\hat{Y}_1(x),\ \hat{Y}_2(x),\ \ldots,\ \hat{Y}_B(x).$$

3. Average all tree predictions:

$$\hat{Y}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{Y}_b(x).$$

Key intuition: a single tree is unstable; many diverse trees averaged together are much more stable. This is the law of large numbers applied to predictions.

In practice: with $B = 2000$ trees and random subsampling, the average tends to converge to a stable function that captures the true signal without overfitting to noise.

[Slide] Random Forest: Illustration

The diagram shows four trees, each giving a different leaf prediction for the same new observation x: \$9,800, \$10,500, \$9,200, \$10,100 (the red paths).

All four predictions are averaged:

$$\hat{Y}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{Y}_b(x)$$

The final prediction is stable even though individual trees disagree.

Variance reduction works here because the tree-specific errors go in different directions and partly cancel when summed. The more diverse the trees (thanks to the two sources of randomness), the more cancellation occurs.

Causal Tree

[Slide] Section Title — Causal Tree

We now adapt the decision tree idea to causal inference. The jump from **random forest** to **causal forest** requires understanding the **causal tree** first.

[Slide] From Prediction Tree to Causal Tree

The difference between a prediction tree and a causal tree is entirely in the **goal** and the **leaf estimate**:

	Prediction tree	Causal tree
Goal	Predict outcome Y_i	Estimate treatment effect $\tau(x)$
Split rule	Find groups with different outcome *levels*	Find groups with different treatment *effects*
Leaf value	$\hat{Y}(\ell) = \bar{Y}_\ell$	$\hat{\tau}(\ell) = \bar{Y}^1_\ell - \bar{Y}^0_\ell$

The structure of the algorithm is identical. What changes is what we are trying to maximize when we choose splits, and what we report in each leaf.

Key analogy: **prediction trees search for outcome heterogeneity; causal trees search for treatment-effect heterogeneity.**

[Slide] Causal Tree — What Does a Leaf Estimate?

In a causal tree, each leaf contains both treated and control observations. The leaf estimate is the **difference in means** between the two groups within that leaf:

$$\hat{\tau}(\ell) = \bar{Y}_{\ell}^{1} - \bar{Y}_{\ell}^{0}$$

The table on the slide shows two leaves:

- **Leaf: Young, low education**: Treated mean = \$11,000, Control mean = \$8,000. Leaf effect = \$3,000.
- **Leaf: Older, high experience**: Treated mean = \$18,500, Control mean = \$18,000. Leaf effect = \$500.

When a new worker with covariates x arrives, we identify which leaf $\ell(x)$ they belong to, and assign them the leaf's estimated treatment effect:

$$\hat{\tau}(x) = \hat{\tau}(\ell(x))$$

Interpretation: **workers with similar characteristics have similar treatment effects**.

[Slide] Causal Tree — Finding the Best Split

The split criterion changes from prediction to causal inference.

- **Prediction tree**: choose splits that make outcomes within each leaf more *homogeneous* (minimize within-leaf variance).
- **Causal tree**: choose splits that make treatment effects *different* across leaves (maximize cross-leaf heterogeneity).

Formally, for a candidate split (X^j, s) defining regions R_1 and R_2:

1. Estimate treatment effects in each region: $\hat{\tau}(R_k) = \bar{Y}_{R_k}^{1} - \bar{Y}_{R_k}^{0}$.
2. Compute the treatment-effect difference:

$$\Delta(j, s) = |\hat{\tau}(R_1) - \hat{\tau}(R_2)|$$

3. Choose the split that **maximizes** $\Delta(j, s)$.

We want to find subgroups with the most different treatment effects — that is where the heterogeneity lives.

[Slide] Causal Tree — A Small Split Example

Look at the table. Three candidate splits for our job training data:

Split	$\hat{\tau}(R_1)$	$\hat{\tau}(R_2)$	Δ
Age < 30	\$3,000	\$600	**\$2,400**
Education < 12	\$2,100	\$1,400	\$700
Prior earnings $< 5,000$	\$2,600	\$1,000	\$1,600

The tree first splits on **Age** < 30 because that split creates the largest treatment-effect difference (\$2,400).

Notice the important distinction: the tree chose age *not* because young and old workers have different wage levels (that would be a prediction tree split) but because they have **different training effects**. Age 30 is the variable that best separates "high-effect" from "low-effect" workers.

[Slide] Causal Tree — Honest Estimation

Here is a subtle but critical problem. The causal tree searches over many possible splits and picks the one with the largest Δ. If we use the **same data** to choose the split and to estimate the treatment effect in the resulting leaves, we will tend to **overstate the treatment effects**. The chosen split looks good partly by chance — it won the search competition — and using the same observations to confirm it leads to biased estimates.

This is called the **double-dipping problem**.

The solution is **honest estimation**: use separate data for the two jobs.

- **Splitting sample**: finds which variables and cutpoints maximize treatment-effect heterogeneity (Δ). This determines the tree structure.
- **Estimation sample**: uses fresh data to compute $\hat{\tau}(\ell) = \bar{Y}_\ell^1 - \bar{Y}_\ell^0$ in each leaf that the splitting sample defined.

The diagram shows this clearly: from a random subsample, split it into two parts; one part decides where to split, the other part estimates the effects.

Why "honest"? The effect is estimated on data that did not influence the choice of split. The split selection and the effect estimation are independent. This is analogous to pre-registration in an RCT: you do not change your hypothesis based on the data.

Honesty is also what allows causal forest to produce **valid confidence intervals** — a key advantage over ad-hoc subgroup analysis.

Causal Forest

[Slide] Section Title — Causal Forest

We now have all the pieces. A **causal forest** is simply a collection of many honest causal trees, with their treatment-effect estimates averaged together.

[Slide] From Random Forest to Causal Forest

The analogy is clean:

	Random Forest	Causal Forest
Building block	Prediction trees	Honest causal trees
Each tree outputs	$\widehat{Y}_b(x)$	$\widehat{\tau}_b(x)$
Final estimate	$\widehat{Y}(x) = \frac{1}{B}\sum_b \widehat{Y}_b(x)$	$\widehat{\tau}(x) = \frac{1}{B}\sum_b \widehat{\tau}_b(x)$

Random forest stabilizes outcome prediction. Causal forest stabilizes CATE estimation. The averaging logic is identical.

[Slide] Causal Forest: Algorithm

Here is the full causal forest algorithm:

1. Draw a random subsample of observations.
2. Split that subsample into a **splitting sample** and an **estimation sample**.
3. Use the splitting sample to grow one honest causal tree (choose splits that maximize treatment-effect heterogeneity Δ).
4. Use the estimation sample to compute leaf effects $\widehat{\tau}(\ell) = \bar{Y}_\ell^1 - \bar{Y}_\ell^0$.
5. Repeat steps 1–4 for B trees (e.g., $B = 2000$).
6. For a worker with characteristics x, average all tree-specific estimates:

$$\widehat{\tau}(x) = \frac{1}{B}\sum_{b=1}^{B} \widehat{\tau}_b(x).$$

In `grf`, you simply call `causal_forest(X, Y, W, num.trees = 2000)` and the package handles all of this internally.

[Slide] Causal Forest: Illustration

The diagram mirrors the random forest illustration, but now the leaves show treatment-effect estimates rather than outcome predictions:

- Tree 1 → \$2,800
- Tree 2 → \$2,200
- Tree $B-1$ → \$3,100
- Tree B → \$2,500

Averaged: $\hat{\tau}(x) = \frac{1}{B}\sum_b \hat{\tau}_b(x)$.

Each tree sees a different subsample and uses a different random feature subset. Their CATE estimates differ, but averaging stabilizes the final estimate.

[Slide] Why Does Causal Forest Help?

The intuition is exactly the same as for random forest:

$$\hat{\tau}(x) = \tau(x) + \frac{1}{B}\sum_{b=1}^{B} \varepsilon_b(x)$$

Each tree-specific error $\varepsilon_b(x)$ has some variance. But if the errors are not perfectly correlated across trees (because each tree used a different subsample), then averaging B trees reduces the variance of the final estimate.

A single causal tree is easy to interpret but noisy. A causal forest trades some interpretability for substantially lower variance and more reliable estimates.

The key message on the slide: "Random forest averages noisy outcome predictions. Causal forest averages noisy treatment-effect estimates."

[Slide] Causal Forest — What Do We Get After Estimation?

After estimating the causal forest, we have:

1. **Individual CATE estimates**: $\hat{\tau}(x_i)$ for each worker in the sample.
2. **ATE**: average the CATE over all observations:

$$\hat{\alpha}_{\text{ATE}} \approx \frac{1}{N}\sum_{i=1}^{N} \hat{\tau}(x_i)$$

3. **ATT**: average the CATE among treated workers:

$$\hat{\alpha}_{\text{ATT}} \approx \frac{1}{N_1}\sum_{i:D_i=1} \hat{\tau}(x_i)$$

4. **Heterogeneity summaries**: the CATE distribution, the Best Linear Projection (BLP), and variable importance.

We will use all of these in the R example.

[Slide] Causal Forest — How to Interpret $\hat{\tau}(x_i)$

A common misunderstanding: $\hat{\tau}(x_i)$ is **not** worker i's true individual treatment effect. The fundamental problem of causal inference still applies — we observe only one potential outcome for each person.

Better interpretation: $\hat{\tau}(x_i)$ **is the estimated average treatment effect for workers with characteristics similar to** x_i.

The example on the slide: if $\hat{\tau}(x_i) = \$2{,}500$, this means that workers who have similar age, education, race, and prior earnings to worker i are estimated to gain about \$2,500 from job training, on average.

This is an important conceptual point. Keep it in mind when discussing results with a non-technical audience. Causal forest gives us CATE — a local average, not a personal counterfactual.

[Slide] After Causal Forest — CATE Distribution

Once we have individual CATE estimates, the first thing to look at is their **distribution**.

The histogram shows all $\hat{\tau}(x_i)$ values. The red dashed line is the mean (close to the ATE).

What does the width of this distribution tell us?

- A **wide** distribution means treatment effects differ substantially across workers. Targeting makes sense — prioritizing high-effect workers would greatly improve cost-effectiveness.
- A **narrow** distribution means most workers have similar estimated effects. Targeting adds little value.

Policy use: the CATE distribution answers the first question: *is there meaningful heterogeneity to exploit?*

[Slide] After Causal Forest — Best Linear Projection (BLP)

The individual $\hat{\tau}(x_i)$ values are informative, but there can be hundreds of them. How do we summarize which worker characteristics are associated with larger effects?

Best Linear Projection (BLP) regresses the estimated CATEs on the centered covariates:

$$\hat{\tau}(x_i) = \beta_0 + \beta_1 \,\text{age}_i + \beta_2 \,\text{educ}_i + \beta_3 \,\text{re74}_i + \cdots + \varepsilon_i$$

Example interpretations: - $\hat{\beta}_1 < 0$: older workers tend to have smaller training effects. - $\hat{\beta}_3 < 0$: workers with higher previous earnings tend to benefit less.

The BLP translates complex, nonparametric CATE estimates into familiar regression-style summaries. In `grf`, call `best_linear_projection(cf, X)`.

Important caveat: BLP captures linear associations. The forest may have found nonlinear patterns that the BLP cannot fully summarize. Use it together with variable importance.

[Slide] After Causal Forest — Variable Importance

Variable importance asks a different question: which covariates does the forest use most often to *split* observations into groups with different treatment effects?

Variable importance is measured as the **fraction of splits** across all trees that use each covariate. A variable with high importance frequently appeared as the best splitter — meaning it reliably separates high-effect from low-effect workers.

From the bar chart on the slide:

1. **Prior earnings** (re74, re75) — highest importance.
2. **Age** — second.
3. **Education** — third.
4. **Married** and **race** — lower.

Important nuance: variable importance tells us *that* a variable matters for heterogeneity, but not *in which direction*. A variable can have high importance because it splits high-effect workers from low-effect workers, but the direction may not be monotone. For direction, use BLP.

In `grf`: `variable_importance(cf)`.

R Example

[Slide] Section Title — R Example

Now let us go through the actual R implementation using the LaLonde job training dataset.

[Slide] LaLonde Job Training Data

We use the classic dataset from LaLonde (1986), which is built into R's `Matching` package.

- **Treatment** D_i: participation in a job training program.
- **Outcome** Y_i: real earnings in 1978.
- **Covariates** X_i: age, education, race indicators (black, Hispanic), marital status, whether the worker has a degree, and earnings in 1974 and 1975.

The original study was a randomized experiment — which means we have a clean identification strategy. Causal forest will use this randomized assignment and ask: **which workers gain more from job training?**

There are 445 observations: 185 treated and 260 control.

[Slide] Data and Program

Everything you need is in `causal_forest.R`. No external data file is needed — the `lalonde` dataset loads directly from the `Matching` package.

Required packages (run once):

```
install.packages(c("grf", "Matching", "ggplot2", "dplyr"))
```

- **grf**: generalized random forests — provides `causal_forest()`, `variable_importance()`, `best_linear_projection()`.
- **Matching**: ships the LaLonde dataset.
- **ggplot2**: for the CATE distribution and variable importance plots.
- **dplyr**: for subgroup summaries.

Key workflow: prepare (Y, D, X) → estimate forest → ATE/ATT → CATE distribution → variable importance → export.

[Slide] Step 1: Package Installation and Data Loading

```
library(grf)
library(Matching)
library(ggplot2)

data(lalonde)
summary(lalonde)
```

After `data(lalonde)`, you have a data frame with 445 rows. The `summary()` call gives you an overview of the variables.

Take a moment to check the variable names: `treat`, `re78`, `age`, `educ`, `black`, `hisp`, `married`, `nodegr`, `re74`, `re75`. Note that the no-degree indicator is `nodegr` — not `nodegree`. Getting column names right is important for the matrix construction in Step 2.

[Slide] Step 2: Data Preparation

```
Y <- lalonde$re78
D <- lalonde$treat
X <- lalonde[, c("age", "educ", "black", "hisp",
                 "married", "nodegr", "re74", "re75")]
X <- as.matrix(X)
```

- `Y`: the outcome variable — real earnings in 1978.
- `D`: the treatment indicator — 1 if received job training, 0 otherwise.
- `X`: the matrix of pre-treatment covariates.

`causal_forest()` requires `X` to be a matrix (not a data frame), so we apply `as.matrix()`.

One important point about variable selection for `X`: include only **pre-treatment** covariates. We exclude `re78` (the outcome) and `treat` (the treatment). Including post-treatment variables would introduce bias.

[Slide] Step 3: Causal Forest Estimation

```
set.seed(2026)

cf <- causal_forest(
  X         = X,
  Y         = Y,
  W         = D,
  num.trees = 2000
)
```

A few notes:

- `W = D`: the `grf` package uses `W` for the treatment variable. Our course notation uses D_i, but they are the same thing.
- `num.trees = 2000`: more trees give more stable estimates. For final results, 2000 is a good default. The computation typically takes less than a minute on the LaLonde dataset.
- `set.seed(2026)`: ensures reproducibility. The forest uses random subsampling and random feature selection, so results vary slightly across runs without a seed.

The package internally handles: honest splitting, random subsampling, random feature selection, and forest averaging. You do not need to code these steps manually.

[Slide] Step 4: Estimate ATE and ATT

```
ate <- average_treatment_effect(cf, target.sample = "all")
att <- average_treatment_effect(cf, target.sample = "treated")

ate
att
```

Each function returns a named vector with `estimate` and `std.err`.

- `target.sample = "all"` → estimates α_{ATE}.
- `target.sample = "treated"` → estimates α_{ATT}.

With `num.trees = 2000` and `set.seed(2026)`, you should get approximately:

- ATE ≈ **$1,583** (SE ≈ $672), 95% CI: [$266, $2,900]
- ATT ≈ **$1,792** (SE ≈ $841), 95% CI: [$143, $3,440]

The ATT is larger than the ATE, which makes sense: the training program was better at enrolling workers who benefit most from it — selection into treatment is consistent with high-gain workers participating.

[Slide] What ATE and ATT Mean Here

Conceptually, the forest first estimates $\hat{\tau}(x_i)$ for every observation, then averages:

$$\hat{\alpha}_{\text{ATE}} \approx \frac{1}{N}\sum_{i=1}^{N}\hat{\tau}(x_i), \qquad \hat{\alpha}_{\text{ATT}} \approx \frac{1}{N_1}\sum_{i:D_i=1}\hat{\tau}(x_i)$$

In practice, `grf` uses **forest weights and influence-function adjustments** to produce estimates that are more efficient than simple averaging. The package estimate is the preferred reported number — do not manually compute the averages and report those.

The key point: the `average_treatment_effect()` function in `grf` produces the same estimands we studied earlier (ATE, ATT), but now estimated through the causal forest machinery.

[Slide] Step 5: Get CATE Estimates

```
tau_hat <- predict(cf)$predictions

summary(tau_hat)
```

`predict(cf)$predictions` returns a vector of length $N = 445$, with one estimated CATE for each observation.

Look at the summary:

- **Mean**: close to the ATE (as expected).
- **SD**: measures the estimated spread of treatment effects across workers.
- **Min/Max**: gives the range of individual estimates.

If the SD is large relative to the mean, there is substantial heterogeneity. If the min is negative for some workers, the forest estimates that those workers are harmed (or simply not helped) by the training.

[Slide] Step 6: Plot CATE Distribution

```
cate_df <- data.frame(tau_hat = tau_hat)

ggplot(cate_df, aes(x = tau_hat)) +
  geom_histogram(bins = 30, fill = "steelblue",
                 color = "white") +
  geom_vline(xintercept = mean(tau_hat),
             color = "red", linetype = "dashed") +
  labs(x = "Estimated treatment effect",
       y = "Count")
```

The red dashed line marks the average of the CATE estimates.

When reading the histogram: the width of the distribution tells you how much heterogeneity the forest detected. A wide histogram means some workers have very large estimated gains and others have small ones — policy targeting could be valuable.

[Slide] Step 6: CATE Distribution (Figure)

The figure shows the actual distribution from the LaLonde data.

Key observations:

- The **mean CATE** (red dashed line) is close to the ATE estimate of $1,583.
- The distribution is **right-skewed** and wide, ranging from negative values up to several thousand dollars.
- The spread confirms that there is **substantial heterogeneity**: some workers benefit far more than others.

This histogram already answers the first policy question: yes, heterogeneity is meaningful. Targeting could make the program substantially more cost-effective.

[Slide] Step 7: Summarize Heterogeneity

```r
# Best linear projection
blp <- best_linear_projection(cf, X)
blp

# Variable importance
vi <- variable_importance(cf)
names(vi) <- colnames(X)
sort(vi, decreasing = TRUE)
```

Read the BLP output as a regression table. Each row corresponds to a covariate. A **negative coefficient on age** means that older workers tend to have smaller training effects. A **negative coefficient on re74** means workers with higher pre-training earnings benefit less.

Note: these are not causal estimates of the covariates — they are descriptive associations with the estimated CATE. Do not overinterpret them.

The variable importance output ranks covariates by how often they were used in forest splits. Covariates at the top of the ranking are the most useful for identifying heterogeneity.

[Slide] Step 7: Variable Importance (Figure)

The figure shows a horizontal bar chart of variable importance.

Key findings from the LaLonde data:

- **Age** has the highest importance, followed by **re74** and **re75** (pre-training earnings in 1974 and 1975).
- Education, marital status, and race have lower importance.

Interpretation: the forest mostly relied on **age** and **prior earnings** to distinguish high-effect workers from low-effect workers. This aligns with economic intuition — younger workers with lower prior earnings have more to gain from human capital investment and have more years to benefit from it.

This is a data-driven finding: we did not pre-specify that age or prior earnings should matter. The forest discovered it.

[Slide] Step 8: Export for Post-Analysis

```r
results <- data.frame(
  id      = 1:nrow(X),
  tau_hat = tau_hat,
  age     = lalonde$age,
  educ    = lalonde$educ,
  D       = lalonde$treat
)

write.csv(results, "cate_results.csv", row.names = FALSE)
```

Exporting the CATE estimates as a CSV allows you to:

- Merge with other datasets for further analysis.
- Use in Stata for summary tables or regression analysis.
- Visualize in any software you prefer.

Common post-analysis: sort workers by `tau_hat`, look at the top quartile vs. bottom quartile, and describe what characteristics the high-effect workers have. This gives a clear policy recommendation about who should be prioritized.

Recommended Resources

[Slide] Recommended Resources

For those who want to go deeper, here are the key references:

1. **Wager and Athey (2018)**: "Estimation and Inference of Heterogeneous Treatment Effects using Random Forests." *JASA*, 113(523), 1228–1242.
   - This is the foundational paper for causal forest. It establishes the asymptotic theory and the honesty condition.
2. **Athey, Tibshirani, and Wager (2019)**: "Generalized Random Forests." *Annals of Statistics*, 47(2), 1148–1178.
   - Extends the framework to a general class of estimation problems. The `grf` package implements this.
3. **`grf` documentation**: https://grf-labs.github.io/grf/
   - The R package vignette and function references. Very readable and contains worked examples.
4. **Athey and Imbens (2019)**: "Machine Learning Methods That Economists Should Know About." *Annual Review of Economics*, 11, 685–725.
   - A broader survey of ML methods in economics, with causal forest as a central example. Good for placing today's lecture in the larger literature.

If you plan to use causal forest in your own research, start with Wager and Athey (2018) for the theory, and the `grf` package documentation for implementation.

End of lecture notes for L13_CausalForest_data_v5